RideFlow: شرح المشروع وعلاقته بمحاضرة التكامل والتشغيل البيني

نبذة عامة عن التطبيق

RideFlow هو نظام موزَّع صغير بنكهة تطبيقات حجز الرحلات، أشبه بتطبيق أوبر من حيث الفكرة العامة: راكب يطلب رحلة، سائق يقبلها وينهيها، ثم تجري عملية فحص احتياليّ ودفع وإشعار. لكنّ الهدف منه ليس بناء منتج حجز رحلات حقيقي أو منافس تجاري، بل بناء مختبر تعليمي صغير يمكن تشغيله ومراقبته وكسره عمداً لفهم كيف تتكامل الأنظمة الموزّعة مع بعضها في الواقع.

يتكوّن النظام من سبع خدمات صغيرة ومستقلة، كل واحدة منها مسؤولة عن وظيفة محددة وتعمل في عملية منفصلة عن البقية: خدمة الواجهة الرسومية التي يتعامل معها الراكب والسائق، وبوّابة حافة (edge) تمثّل نقطة الدخول الوحيدة من خارج الشبكة، وبوّابة واجهات (gateway) تتولى التحقق من الهوية وتحديد معدّل الطلبات، وخدمة الرحلات التي تدير دورة حياة الرحلة من الطلب إلى الإكمال، وخدمة فحص الاحتيال، وخدمة الدفع، وخدمة الإشعارات. إلى جانب هذه الخدمات السبع، يعتمد النظام على وسيط رسائل (RabbitMQ) لنقل الأحداث بين الخدمات، وعلى أداة تتبّع موزَّع (Jaeger) لمتابعة رحلة الطلب الواحد عبر كل هذه الخدمات.

جميع هذه الخدمات تُشغَّل معاً عبر Docker Compose، أي أن كل خدمة تعمل داخل حاوية (container) منفصلة ومستقلة، وتتواصل مع بقية الخدمات عبر الشبكة فقط، تماماً كما لو كانت أنظمة مختلفة تعود لفرق أو شركات مختلفة. هذا التفصيل ليس مجرد وسيلة تشغيل مريحة، بل هو جزء من صلب الفكرة: لأن الخدمات تعيش في حاويات منعزلة ولا تتشارك الذاكرة أو الكود، فإن أي تواصل بينها يجب أن يمرّ عبر الشبكة بشكل صريح، بنفس الطريقة التي تتواصل بها الأنظمة الحقيقية المستقلة في أي شركة. لذلك من المفيد ذكر Docker Compose عند شرح المشروع، لأنه يبرز أن هذه ليست وحدات برمجية داخل برنامج واحد، بل أنظمة منفصلة فعلاً تحتاج تكاملاً حقيقياً لتعمل معاً.

الفكرة والهدف من المشروع

نقطة الانطلاق في المحاضرة هي أنه لا يوجد نظام مؤسساتي يعيش وحده، وأن قيمة أي نظام تأتي من الأنظمة الأخرى التي يتحدّث إليها، وأن الحدود بين الأنظمة هي المكان الذي تتركّز فيه معظم الأعطال، كما تُظهر دراسة حالة موقع HealthCare.gov عام 2013، حيث نجح كل جزء من النظام بمفرده لكن فشل الربط بين الأجزاء لأن أحداً لم يكن يملك هذه الحدود فعلياً.

انطلاقاً من هذه الفكرة، صُمِّم RideFlow ليكون تجسيداً عملياً لمفاهيم المحاضرة بدلاً من أن تبقى نظرية فقط. كل مفهوم من مفاهيم التكامل والتشغيل البيني الذي تناولته المحاضرة له مكان محدد وملموس داخل هذا النظام، بحيث يمكن تشغيله، أو تعطيله عمداً، أو قياس أثره بالأرقام الحقيقية بدلاً من الاكتفاء بفهمه كفكرة مجردة على شريحة عرض. لهذا السبب أيضاً يحتوي المشروع على غرفة تحكّم كاملة تتيح تعطيل أي خدمة أو وصلة بين الخدمات عمداً، ولوحة مراقبة حيّة تعرض ما يحدث للنظام لحظة بلحظة أثناء هذا التعطيل، لأن الهدف الأساسي هو تمكين تجربة عملية حقيقية حول أحد أبعاد التكامل، وليس مجرد عرض نظام يعمل بشكل سليم فقط.

علاقة المشروع بمحاضرة التكامل والتشغيل البيني

يمكن الربط بين كل عنصر رئيسي من المحاضرة وموقعه الفعلي داخل RideFlow كما يلي.

فيما يخص أنماط التكامل التي تناولتها المحاضرة، نجد بوّابة الواجهات ممثَّلة حرفياً بخدمة gateway التي توفر مدخلاً واحداً للمصادقة وتحديد المعدّل والتوجيه، ونجد نمط الوسيط وناقل الأحداث ممثَّلاً بوسيط الرسائل RabbitMQ الذي ينقل الأحداث بين خدمة الرحلات وخدمة الدفع وخدمة الإشعارات دون أن تعرف هذه الخدمات بعضها بعضاً بشكل مباشر، ونجد أيضاً نمط المراسلة عبر Webhook ممثَّلاً بخدمة الإشعارات التي ترسل إشعاراً موقَّعاً إلى تطبيق السائق عند اكتمال الدفع.

وفيما يخص التزامن مقابل اللاتزامن، يحتوي النظام على مثالين متناقضين عمداً ليمكن مقارنتهما: الاتصال بين خدمة الدفع وخدمة فحص الاحتيال اتصال متزامن تقليدي من نوع طلب واستجابة، حيث تنتظر خدمة الدفع رداً فورياً من خدمة فحص الاحتيال قبل أن تكمل عملها، بينما الاتصال عبر طوابير RabbitMQ بين خدمة الرحلات وخدمة الدفع، وبين خدمة الدفع وخدمة الإشعارات، اتصال لاتزامني بالكامل، حيث تنشر الخدمة المرسِلة الحدث وتمضي في عملها دون انتظار أن تعالجه الخدمة المستقبِلة فوراً.

وفيما يخص أساليب الواجهات البرمجية، تستخدم خدمة الرحلات أسلوب REST بعقد موصوف مسبقاً بصيغة OpenAPI، بينما تستخدم خدمة الإشعارات أسلوب Webhook حيث تبادر الخدمة نفسها بإرسال رسالة موقّعة بدلاً من انتظار أن يستطلعها طرف آخر.

وفيما يخص طبقات التشغيل البيني التي تحدثت عنها المحاضرة، بدءاً من الطبقة التقنية وصولاً إلى الطبقة التنظيمية، يظهر ذلك في النظام عبر وجود قناة اتصال محددة لكل وصلة بين الخدمات مثل HTTP أو بروتوكول الرسائل، وصيغة رسائل متفق عليها بصيغة JSON موصوفة بمخطط واضح، ومعنى موحّد لكل حقل بحيث تفهم خدمة الدفع وخدمة الإشعارات ما تعنيه كلمة "رحلة" أو "مبلغ" بنفس الطريقة تماماً التي تفهمها بها خدمة الرحلات، وأخيراً سياسات تنظيمية مثل التحقق من الهوية عبر JWT وتحديد معدّل الطلبات لكل مستخدم.

وفيما يخص العقود والإصدارات، تتبع خدمة الرحلات مبدأ كتابة العقد أولاً قبل الكود، حيث يوجد عقد OpenAPI لإصدار أول مجمَّد لا يُكسر، وعقد آخر لإصدار ثانٍ يحتوي عمداً على تغيير كاسر لتوضيح الفرق بين تطوّر آمن للواجهة وتطوّر يكسر الأطراف الأخرى، إلى جانب اختبار عقد فعلي يكشف أي كسر قبل أن يصل إلى مستهلك حقيقي.

وفيما يخص أنماط الصمود التي تحمي التكامل من الفشل، يطبّق النظام فعلياً نمط صندوق الإرسال في خدمة الرحلات لضمان نشر الأحداث بأمان حتى عند حدوث انهيار مفاجئ، ويطبّق في خدمة الدفع مفتاح عدم تكرار الأثر لمنع خصم مبلغ مرتين عند تكرار نفس الطلب، وإعادة المحاولة بتراجع زمني متصاعد، وقاطع دارة يفصل الاتصال عن خدمة فحص الاحتيال عند تعثّرها بدل الاستمرار في إرسال طلبات لها، وطابوراً مخصصاً للرسائل التي تعذّرت معالجتها نهائياً حتى لا تُفقد ولا تُعطّل بقية النظام.

وفيما يخص فكرة مراقبة الوصلة نفسها وليس فقط طرفيها، تقوم بذلك خدمة monitor التي تراقب عمق الطوابير وحالة قاطع الدارة ومقاييس كل خدمة، وتُعرض هذه المعطيات حيّة في لوحة المراقبة، تطبيقاً لفكرة أن الأعطال غالباً ما تظهر في الوصلة بين الأنظمة لا داخل أي نظام بمفرده.

وأخيراً، فإن درس دراسة الحالة التي ناقشتها المحاضرة حول ضرورة أن يملك أحد ما مسؤولية الحدود بين الأنظمة، ينعكس في أن كل وصلة بين خدمتين داخل RideFlow لها مالك واضح ومحدد، هو الخدمة التي عرّفت الطابور أو العقد المستخدم في تلك الوصلة، بحيث لا تبقى أي وصلة بلا طرف مسؤول عنها كما حدث في الحالة التي تناولتها المحاضرة.

خلاصة

RideFlow إذاً ليس تطبيق حجز رحلات بالمعنى التجاري، بل هو نظام موزَّع بُني خصيصاً ليكون تطبيقاً عملياً لمحاضرة التكامل والتشغيل البيني، بحيث يمكن لأي شخص أن يشغّله عبر Docker Compose، ويرى فيه كل مفهوم من مفاهيم المحاضرة كشيء يعمل فعلياً أمامه بدل أن يبقى فكرة نظرية، وأن يختار أحد أبعاد التكامل، ويعطّل الطرف الآخر من الوصلة عمداً، ويقيس ما يحدث للمعطيات بأرقام حقيقية.